Evaluación perezosa y recursión

Laboratorio 05: extensión de MiniLisp++
Notas 13 y 14

M. en C.I.C. Manuel Soto Romero

Facultad de Ciencias, UNAM

Semestre 2027-1

Punto de partida: laboratorio 04

Se conserva

La nueva versión mantiene números naturales, booleanos, primitivas, `let`, `let*`, funciones con varios parámetros, aplicaciones con varios argumentos y alcance estático mediante cerraduras.

Se añade

- `if`, `cond` y `letrec` en la sintaxis fuente;
- evaluación perezosa mediante cerraduras de expresión;
- puntos estrictos para exigir valores; y
- recursión mediante el combinador de punto fijo Y.

Una sola estrategia

En este laboratorio no se implementa un selector de estrategias: MiniLisp++ será perezoso en todas sus evaluaciones.

Nueva regla para el código

A partir de este laboratorio

Después de revisar el código de prácticas anteriores, queda prohibido usar la notación `do` y expresiones `case ... of` en el código que escribas.

Código que puede razonarse ecuacionalmente

Define las funciones mediante ecuaciones, patrones en el lado izquierdo, guardias, funciones auxiliares, recursión, composición y expresiones `let`/`in`. Debes poder explicar cada reducción reemplazando iguales por iguales.

Alcance de la restricción

La restricción se aplica a `Interp.hs`, `MiniLispPlusPlus.hs` y cualquier auxiliar que agregues. No se revisan `Lexer.hs` ni `Grammars.hs`, pues Alex y Happy los generan automáticamente. La infraestructura y las pruebas proporcionadas tampoco forman parte de tu implementación.

Reto 1: reconocer las nuevas palabras

Repaso breve

Las palabras reservadas deben aparecer antes que la regla general de identificadores. La coincidencia más larga permite conservar nombres como `ifx`, `cond1` o `letrecAux`.

Comportamiento esperado

```
lexer "if cond else letrec ifx"
==
[ TokenIf, TokenCond, TokenElse,
  TokenLetRec, TokenId "ifx" ]
```

Reto

Completa en `Lexer.x` las reglas para `if`, `cond`, `else` y `letrec`. Conserva sin cambios las formas heredadas del laboratorio 4.

Reto 2: extender el ASA fuente

Nuevas formas

$$
\begin{array}{rcl}
e & ::= & \cdots \mid (\text{if } e\ e\ e) \\
 & \mid & (\text{cond } (e\ e)^{+}\ (\text{else } e)) \\
 & \mid & (\text{letrec } (x\ e)\ e).
\end{array}
$$

Un `cond` contiene al menos una cláusula ordinaria y siempre termina con exactamente una cláusula `else`.

ASA esperado

```
parse (lexer
  "(cond (#f 1) ((not #f) 2) (else 3))")
==
CondS [(BooleanS False, NumS 1),
       (NotS (BooleanS False), NumS 2)]
      (NumS 3)
```

Reto

Completa en `Grammars.y` las producciones de `IfS`, `CondS` y `LetRecS`. Diseña `Clauses` de manera que no pueda reconocerse un `cond` sin `else`.

Reto 3: `cond` es azúcar sintáctica

Anidamiento hacia la derecha

$$\text{cond } (c_1, e_1)\ (c_2, e_2) \text{ else } e_3$$
$$\rightsquigarrow \mathit{If}(c_1, e_1, \mathit{If}(c_2, e_2, e_3)).$$

Se conserva el orden de las cláusulas y el `else` se convierte en la última alternativa. El núcleo no contiene un constructor `Cond`.

Ejemplo

```
desugar
  (CondS [(BooleanS False, NumS 1),
          (BooleanS True, NumS 2)]
         (NumS 3))
==
Just (If (Boolean False) (Num 1)
      (If (Boolean True) (Num 2) (Num 3)))
```

Reto

Implementa `desugarCond` y extiende `desugar`. Cada condición, rama y alternativa debe desazucararse también.

Reto 3: `letrec` se expresa con Y

Transformación

$$\mathcal{D}(\mathit{LetRecS}(f, s_1, s_2)) = \mathcal{D}\big(\mathit{LetS}(f, \mathit{AppS}(\mathit{IdS}(\mathtt{Y}), \mathit{FunS}([f], s_1)), s_2)\big).$$

Después se elimina el `let` con la transformación ya conocida. El núcleo no contiene un constructor `LetRec`.

Lectura

`FunS [f] s1` recibe como parámetro la función recursiva. El combinador Y obtiene su punto fijo y la asociación con `f` queda disponible en `s2`.

Reto

Añade el caso `LetRecS` a `desugar`. No implementes ambientes recursivos ni utilices Z: esta versión es perezosa y debe usar Y.

Reto 4: dos clases de cerraduras

Valores

```
data Value
  = NumV Int
  | BooleanV Bool
  | ClosureV Nombre ASA Env
  | ExprV ASA Env
```

`ClosureV` guarda el ambiente donde se define una función. `ExprV` guarda una expresión sin evaluar y el ambiente necesario para interpretarla después.

Aplicación perezosa

Al aplicar una cerradura $\langle p, b, \varepsilon_f \rangle$, el argumento a no se evalúa. El cuerpo se evalúa en

$$\varepsilon_f[p \leftarrow \langle a, \varepsilon \rangle],$$

donde ε es el ambiente de la llamada.

Alcance estático

El cuerpo usa el ambiente de definición de la función. El ambiente de la llamada se conserva únicamente dentro de la cerradura de expresión del argumento.

Reto 4: puntos estrictos

Consultar no significa exigir

$$\frac{\varepsilon(x) = w}{Id(x), \varepsilon \Rightarrow w}.$$

Si w es una cerradura de expresión, la regla del identificador la devuelve sin evaluar su contenido.

Función strict

$$\frac{e, \varepsilon \Rightarrow w \qquad \text{strict}(w) = v}{\text{strict}(\langle e, \varepsilon \rangle) = v} \qquad \frac{v \text{ no es una cerradura de expresión}}{\text{strict}(v) = v}.$$

Dónde se aplica

Son puntos estrictos los operandos de `+`, `-` y `not`, la condición de `if` y la posición de función de una aplicación. `if` exige la condición y evalúa solamente la rama seleccionada.

Reto 4: implementar el intérprete

Regla de aplicación

$$\frac{f, \varepsilon \Rightarrow w_f \quad \text{strict}(w_f) = \langle p, b, \varepsilon_f \rangle \quad b, \varepsilon_f[p \leftarrow \langle a, \varepsilon \rangle] \Rightarrow w}{App(f, a), \varepsilon \Rightarrow w}.$$

La estrategia debe ser observable

```
bigStep []
  (App (Fun "x" (Num 7)) (Id "libre"))
==
Just (NumV 7)
```

Funciones por completar

```
lookupEnv :: Nombre -> Env -> Maybe Value
strict    :: Value -> Maybe Value
bigStep   :: Env -> ASA -> Maybe Value
```

No exijas una asociación en `lookupEnv`. Los puntos estrictos son responsables de solicitar los valores que necesitan.

Reto 5: integrar Y

Combinador clásico

$$Y = \lambda f.(\lambda x.f(xx))(\lambda x.f(xx)).$$

Represéntalo mediante los constructores `Fun`, `App` e `Id`. Evalúalo una vez y liga el valor resultante al nombre `Y` en `prelude`.

Recursión observable con booleanos

```
(letrec
  (f (lambda (b)
       (if b 7 (f #t))))
  (f #f))
==> NumV 7
```

Interfaz pública

```
combinadorY :: ASA
prelude     :: Env
evalua      :: String -> Maybe Value
```

`evalua` inicia en `prelude` y aplica `strict` al resultado final. No agregues Y al lexer ni al parser: es un identificador ligado por el ambiente inicial.

Entrega y comprobación

Entrega

Entrega `Lexer.x`, `Grammars.y`, `Interp.hs` y `MiniLispPlusPlus.hs`. No modifiques `TestLaboratorio05.hs`.

Compilación

```
alex Lexer.x
happy --ghc Grammars.y
ghc -package QuickCheck -package haskeline \
    TestLaboratorio05.hs -o pruebas
./pruebas
```

Antes de entregar

Verifica que las pruebas pasen y revisa manualmente que tu código no use `do` ni `case ... of`. Debes poder explicar ecuacionalmente el desazucarado, la creación y exigencia de `ExprV`, el alcance estático y la ejecución de Y.